BIG-M Method

Problem Formulation: Consider a production planning scenario where a firm must determine the quantities of two products, x1 and x2, to manufacture in order to minimize total cost, represented by the objective function Z = 4x1 + x2. The production process is subject to three constraints: an exact material-balance requirement (3x1 + x2 = 3), a minimum output requirement to meet contractual demand (4x1 + 3x2 ≥ 6), and a resource-capacity limit (x1 + 2x2 ≤ 4), with both decision variables restricted to non-negative values. Since the model contains an equality constraint and a "greater-than-or-equal-to" constraint, artificial variables must be introduced to obtain an initial basic feasible solution, and a large penalty cost (M) is assigned to these artificial variables in the objective function to force them out of the optimal solution. This problem is solved using the Big-M Simplex Method, iterating the simplex tableau until all Zj − Cj values are non-positive, at which point the optimal values of x1, x2, and the minimum objective value Z are obtained.

Code Part

```
"""
BIG-M METHOD (Penalty Method) - Linear Programming Solver
------------------------------------------------------------
Problem (a well-known Big-M textbook example):

    Minimize   Z = 4x1 + x2

    Subject to:
        3x1 +  x2  = 3        (equality constraint  -> needs artificial var)
        4x1 + 3x2 >= 6        (>= constraint        -> needs surplus + artificial var)
         x1 + 2x2 <= 4        (<= constraint        -> needs slack var)
        x1, x2 >= 0

Standard form after adding slack (S), surplus (S), and artificial (A) variables:

    3x1 +  x2 + A1               = 3
    4x1 + 3x2      - S1      + A2 = 6
     x1 + 2x2           + S2     = 4

Big-M objective (minimization):
    Z = 4x1 + x2 + 0.S1 + 0.S2 + M.A1 + M.A2
"""
```

```python
from fractions import Fraction as F

M = 1_000_000  # a sufficiently large penalty (Big-M), used only for ranking, kept symbolic via large int

# Variable order in the tableau:
#   x1  x2  S1  S2  A1  A2  |  RHS
var_names = ["x1", "x2", "S1", "S2", "A1", "A2"]

# Cost (objective) coefficients for MINIMIZATION
c = [4, 1, 0, 0, M, M]

# Initial tableau rows: [x1, x2, S1, S2, A1, A2, RHS]
# Row 1 : 3x1 +  x2         + A1        = 3
# Row 2 : 4x1 + 3x2  - S1        + A2   = 6
# Row 3 :  x1 + 2x2       + S2          = 4
tableau = [
    [3, 1, 0, 0, 1, 0, 3],
    [4, 3, -1, 0, 0, 1, 6],
    [1, 2, 0, 1, 0, 0, 4],
]

# Basic variables initially in each row (indices into var_names)
basis = [4, 5, 3]   # A1, A2, S2


def print_tableau(tableau, basis, c, iteration):
    print(f"\n{'=' * 90}")
    print(f"ITERATION {iteration}")
    print(f"{'=' * 90}")
    header = f"{'Basis':>8}" + "".join(f"{v:>12}" for v in var_names) + f"{'RHS':>12}"
    print(header)
    for i, row in enumerate(tableau):
        row_str = f"{var_names[basis[i]]:>8}" + "".join(f"{fmt(val):>12}" for val in row[:-1]) + f"{fmt(row[-1]):>12}"
        print(row_str)
```

```python
        print(row_str)

    # Compute Zj row and Zj - Cj row
    zj = []
    for j in range(len(var_names)):
        s = sum(c[basis[i]] * tableau[i][j] for i in range(len(tableau)))
        zj.append(s)
    zj_rhs = sum(c[basis[i]] * tableau[i][-1] for i in range(len(tableau)))

    cj_row = "Cj      " + "".join(f"{fmt(c[j]):>12}" for j in range(len(var_names)))
    print(cj_row)

    zj_row = "Zj      " + "".join(f"{fmt(zj[j]):>12}" for j in range(len(var_names))) + f"{fmt(zj_rhs):>12}"
    print(zj_row)

    zc_row_vals = [zj[j] - c[j] for j in range(len(var_names))]
    zc_row = "Zj-Cj    " + "".join(f"{fmt(v):>12}" for v in zc_row_vals)
    print(zc_row)

    return zj, zc_row_vals


def fmt(x):
    """Nicely format Fractions / big-M expressions for display."""
    if isinstance(x, F):
        if x.denominator == 1:
            return str(x.numerator)
        return f"{float(x):.3f}"
    return str(x)
```

```python

def to_fraction_tableau(tab):
    return [[F(val) for val in row] for row in tab]


def big_m_simplex(tableau, basis, c):
    tableau = to_fraction_tableau(tableau)
    iteration = 0
    while True:
        zj, zc = print_tableau(tableau, basis, c, iteration)

        # Optimality check (minimization): stop when all Zj - Cj <= 0
        entering = None
        best = 0
        for j in range(len(var_names)):
            if zc[j] > best:
                best = zc[j]
                entering = j

        if entering is None:
            print("\nOptimality reached: all (Zj - Cj) <= 0.")
            break

        print(f"\nEntering variable : {var_names[entering]}  (largest positive Zj-Cj = {fmt(best)})")

        # Ratio test (minimum ratio, only positive entries in entering column)
        ratios = []
        for i in range(len(tableau)):
            col_val = tableau[i][entering]
            if col_val > 0:
                ratios.append((tableau[i][-1] / col_val, i))
            else:
                ratios.append((None, i))

```

```python

        valid_ratios = [(r, i) for r, i in ratios if r is not None]
        if not valid_ratios:
            print("Problem is UNBOUNDED.")
            return None, None

        min_ratio, leaving_row = min(valid_ratios, key=lambda t: t[0])
        print(f"Leaving variable  : {var_names[basis[leaving_row]]}  (min ratio = {fmt(min_ratio)})")

        # Pivot
        pivot_val = tableau[leaving_row][entering]
        tableau[leaving_row] = [val / pivot_val for val in tableau[leaving_row]]
        for i in range(len(tableau)):
            if i != leaving_row:
                factor = tableau[i][entering]
                if factor != 0:
                    tableau[i] = [tableau[i][j] - factor * tableau[leaving_row][j] for j in range(len(tableau[i]))]

        basis[leaving_row] = entering
        iteration += 1

    return tableau, basis


if __name__ == "__main__":
    print("BIG-M SIMPLEX METHOD")
    print("Minimize Z = 4x1 + x2")
    print("s.t.  3x1 +  x2      = 3")
    print("      4x1 + 3x2     >= 6")
    print("       x1 + 2x2     <= 4")
    print("      x1, x2 >= 0")

    final_tableau, final_basis = big_m_simplex(tableau, basis, c)

```

```python

    if final_tableau is not None:
        print(f"\n{'=' * 90}")
        print("OPTIMAL SOLUTION")
        print(f"{'=' * 90}")
        solution = {v: 0 for v in var_names}
        for i, b in enumerate(final_basis):
            solution[var_names[b]] = final_tableau[i][-1]

        for v in ["x1", "x2"]:
            print(f"  {v} = {fmt(solution[v])}")

        z_value = 4 * solution["x1"] + 1 * solution["x2"]
        print(f"\n  Optimal Objective Value  Z = 4x1 + x2 = {fmt(z_value)}")

        # sanity check: no artificial variable should remain in the basis with positive value
        art_in_basis = any(var_names[b] in ("A1", "A2") and final_tableau[i][-1] != 0
                           for i, b in enumerate(final_basis))
        if art_in_basis:
            print("\n  WARNING: an artificial variable remained positive in the basis -> INFEASIBLE.")
        else:
            print("\n  Feasibility check passed (no artificial variable remains positive).")

```

OUTPUT

```
BIG-M SIMPLEX METHOD
Minimize Z = 4x1 + x2
s.t.  3x1 +  x2      = 3
      4x1 + 3x2     >= 6
       x1 + 2x2     <= 4
      x1, x2 >= 0

==========================================================================================
ITERATION 0
==========================================================================================
   Basis          x1          x2          S1          S2          A1          A2         RHS
      A1           3           1           0           0           1           0           3
      A2           4           3          -1           0           0           1           6
      S2           1           2           0           1           0           0           4
Cj                 4           1           0           0     1000000     1000000
Zj           7000000     4000000    -1000000           0     1000000     1000000     9000000
Zj-Cj        6999996     3999999    -1000000           0           0           0

Entering variable : x1  (largest positive Zj-Cj = 6999996)
Leaving variable  : A1  (min ratio = 1)

==========================================================================================
ITERATION 1
==========================================================================================
   Basis          x1          x2          S1          S2          A1          A2         RHS
      x1           1       0.333           0           0       0.333           0           1
      A2           0       1.667          -1           0      -1.333           1           2
      S2           0       1.667           0           1      -0.333           0           3
Cj                 4           1           0           0     1000000     1000000
Zj                 4     1666668    -1000000           0    -1333332     1000000     2000004
Zj-Cj              0     1666667    -1000000           0    -2333332           0
```

```
Entering variable : x2  (largest positive Zj-Cj = 1666667)
Leaving variable  : A2  (min ratio = 1.200)

================================================================================================
ITERATION 2
================================================================================================
   Basis          x1          x2          S1          S2          A1          A2         RHS
      x1           1           0       0.200           0       0.600      -0.200       0.600
      x2           0           1      -0.600           0      -0.800       0.600       1.200
      S2           0           0           1           1           1          -1           1
Cj                 4           1           0           0     1000000     1000000
Zj                 4           1       0.200           0       1.600      -0.200       3.600
Zj-Cj              0           0       0.200           0 -999998.400-1000000.200

Entering variable : S1  (largest positive Zj-Cj = 0.200)
Leaving variable  : S2  (min ratio = 1)

================================================================================================
ITERATION 3
================================================================================================
   Basis          x1          x2          S1          S2          A1          A2         RHS
      x1           1           0           0      -0.200       0.400           0       0.400
      x2           0           1           0       0.600      -0.200           0       1.800
      S1           0           0           1           1           1          -1           1
Cj                 4           1           0           0     1000000     1000000
Zj                 4           1           0      -0.200       1.400           0       3.400
Zj-Cj              0           0           0      -0.200 -999998.600    -1000000

Optimality reached: all (Zj - Cj) <= 0.
```

```
 x1 = 0.400
 x2 = 1.800

 Optimal Objective Value  Z = 4x1 + x2 = 3.400

 Feasibility check passed (no artificial variable remains positive).
```

Transportation Problem

Problem Formulation: Consider a manufacturing company operating three factories (S1, S2, S3) with production capacities of 50, 60, and 50 units respectively, that must supply goods to four regional warehouses (D1, D2, D3, D4) with demands of 30, 40, 50, and 40 units respectively. Since total supply (160 units) equals total demand (160 units), this is a balanced transportation problem. The per-unit transportation cost from each factory to each warehouse is given in the cost matrix. The objective is to determine the shipment quantities from each factory to each warehouse that satisfy all supply and demand constraints while minimizing the total transportation cost. This problem is first solved using Vogel's Approximation Method (VAM) to obtain an initial basic feasible solution, which is then tested for optimality and improved using the MODI (Modified Distribution) Method until the optimal shipment plan is reached.

Code Part

```
"""
Transportation Problem: Vogel's Approximation Method (VAM) + MODI Method
------------------------------------------------------------------------
Case study: A company has 3 factories (sources) supplying goods to
4 warehouses (destinations).

Supply (Factories) : S1 = 50, S2 = 60, S3 = 50   -> total = 160
Demand (Warehouses): D1 = 30, D2 = 40, D3 = 50, D4 = 40 -> total = 160
(Balanced transportation problem, since total supply = total demand)

Unit transportation costs (Cost[i][j] = cost to ship 1 unit from
source i to destination j):

        D1   D2   D3   D4
    S1 [  4,   6,   8,   8 ]
    S2 [  6,   8,   6,   7 ]
    S3 [  5,   7,   6,   8 ]

Step 1: Use VAM to get an Initial Basic Feasible Solution (IBFS)
Step 2: Use MODI to test optimality and iteratively improve the
        allocation until the optimal (minimum-cost) solution is found.
"""

```

```python
import copy

# ---------------------------------------------------------------------
# Problem data
# ---------------------------------------------------------------------
supply_orig = [50, 60, 50]
demand_orig = [30, 40, 50, 40]
cost = [
    [4, 6, 8, 8],
    [6, 8, 6, 7],
    [5, 7, 6, 8],
]
source_names = ["S1", "S2", "S3"]
dest_names = ["D1", "D2", "D3", "D4"]


def print_table(allocation, cost, title):
    rows, cols = len(cost), len(cost[0])
    print(f"\n{title}")
    header = "        " + "".join(f"{d:>10}" for d in dest_names)
    print(header)
    for i in range(rows):
        row_str = f"{source_names[i]:<8}"
        for j in range(cols):
            if allocation[i][j] > 0:
                row_str += f"{str(allocation[i][j])+'('+str(cost[i][j])+')':>10}"
            else:
                row_str += f"{'-('+str(cost[i][j])+')':>10}"
        print(row_str)


def total_cost(allocation, cost):
    return sum(allocation[i][j] * cost[i][j]
               for i in range(len(cost)) for j in range(len(cost[0])))


```

```python


# ---------------------------------------------------------------------
# STEP 1: Vogel's Approximation Method (VAM)
# ---------------------------------------------------------------------
def vogel_approximation_method(supply, demand, cost):
    supply = supply.copy()
    demand = demand.copy()
    rows, cols = len(supply), len(demand)
    allocation = [[0] * cols for _ in range(rows)]
    row_done = [False] * rows
    col_done = [False] * cols

    step = 1
    while sum(supply) > 0 and sum(demand) > 0:
        row_penalty = [-1] * rows
        col_penalty = [-1] * cols
```

```
        for i in range(rows):
            if row_done[i]:
                continue
            costs_row = [cost[i][j] for j in range(cols) if not col_done[j]]
            if len(costs_row) >= 2:
                s = sorted(costs_row)
                row_penalty[i] = s[1] - s[0]
            elif len(costs_row) == 1:
                row_penalty[i] = costs_row[0]

        for j in range(cols):
            if col_done[j]:
                continue
            costs_col = [cost[i][j] for i in range(rows) if not row_done[i]]
            if len(costs_col) >= 2:
                s = sorted(costs_col)
                col_penalty[j] = s[1] - s[0]
            elif len(costs_col) == 1:
                col_penalty[j] = costs_col[0]

        max_row_pen = max(row_penalty)
        max_col_pen = max(col_penalty)

        if max_row_pen >= max_col_pen:
            i = row_penalty.index(max_row_pen)
            j = min((j for j in range(cols) if not col_done[j]),
                    key=lambda j: cost[i][j])
        else:
            j = col_penalty.index(max_col_pen)
            i = min((i for i in range(rows) if not row_done[i]),
                    key=lambda i: cost[i][j])

        qty = min(supply[i], demand[j])
        allocation[i][j] = qty
        print(f"Step {step}: Allocate {qty} units to "
              f"({source_names[i]} -> {dest_names[j]}), cost/unit = {cost[i][j]}")
        step += 1

        supply[i] -= qty
        demand[j] -= qty
        if supply[i] == 0:
            row_done[i] = True
        if demand[j] == 0:
            col_done[j] = True

    return allocation


# ---------------------------------------------------------------------
# STEP 2: MODI (Modified Distribution) Method
# ---------------------------------------------------------------------
def find_closed_loop(start, basic_cells):
    """Find a closed loop (alternating row/column moves) starting and
    ending at `start`, passing only through cells in basic_cells."""
    all_cells = list(basic_cells)
    if start not in all_cells:
        all_cells.append(start)

```

```
133
134         def search(path, horizontal):
135             last = path[-1]
136             candidates = [c for c in all_cells if c != last and
137                           ((c[0] == last[0]) if horizontal else (c[1] == last[1]))]
138             for nxt in candidates:
139                 if nxt == start and len(path) >= 3:
140                     return path + [nxt]
141                 if nxt in path:
142                     continue
143                 result = search(path + [nxt], not horizontal)
144                 if result:
145                     return result
146             return None
147
148         return search([start], True)
149
150
151     def modi_method(supply, demand, cost, allocation):
152         rows, cols = len(supply), len(demand)
153         allocation = copy.deepcopy(allocation)
154         iteration = 1
155
156         while True:
157             basic_cells = [(i, j) for i in range(rows) for j in range(cols)
158                            if allocation[i][j] > 0]
159             required = rows + cols - 1
160
161             # Handle degeneracy: add a zero-allocation basic cell if needed
162             if len(basic_cells) < required:
163                 for i in range(rows):
164                     for j in range(cols):
165                         if (i, j) not in basic_cells:
166                             trial = basic_cells + [(i, j)]
167                             if find_closed_loop((i, j), basic_cells) is None:
168                                 basic_cells.append((i, j))
169                                 allocation[i][j] = 0  # epsilon ~ 0
170                                 break
```

```
                    break
                if len(basic_cells) == required:
                    break

        # Compute u_i, v_j potentials (u_1 = 0)
        u = [None] * rows
        v = [None] * cols
        u[0] = 0
        changed = True
        while changed:
            changed = False
            for (i, j) in basic_cells:
                if u[i] is not None and v[j] is None:
                    v[j] = cost[i][j] - u[i]
                    changed = True
                elif v[j] is not None and u[i] is None:
                    u[i] = cost[i][j] - v[j]
                    changed = True

        print(f"\n--- MODI Iteration {iteration} ---")
        print("u =", [f"{source_names[i]}:{u[i]}" for i in range(rows)])
        print("v =", [f"{dest_names[j]}:{v[j]}" for j in range(cols)])

        # Opportunity cost for non-basic cells
        opp_cost = {}
        for i in range(rows):
            for j in range(cols):
                if (i, j) not in basic_cells:
                    opp_cost[(i, j)] = cost[i][j] - (u[i] + v[j])

        print("Opportunity costs (non-basic cells):")
        for (i, j), val in opp_cost.items():
            print(f"  {source_names[i]}-{dest_names[j]}: {val}")

        if not opp_cost or min(opp_cost.values()) >= 0:
            print("\nAll opportunity costs >= 0  ->  Optimal solution reached.")
            break

```

```
        # Entering cell = most negative opportunity cost
        enter = min(opp_cost, key=opp_cost.get)
        print(f"Entering cell: {source_names[enter[0]]}-{dest_names[enter[1]]} "
              f"(opportunity cost = {opp_cost[enter]})")

        loop = find_closed_loop(enter, basic_cells)
        print("Closed loop:", [f"{source_names[i]}-{dest_names[j]}" for (i, j) in loop])

        minus_cells = loop[1::2][:-1] if loop[-1] == loop[0] else loop[1::2]
        # loop returned as [start, ..., start]; drop duplicate end for processing
        loop_cells = loop[:-1]
        plus_cells = loop_cells[0::2]
        minus_cells = loop_cells[1::2]

        theta = min(allocation[i][j] for (i, j) in minus_cells)
        print(f"Theta (max units to reallocate) = {theta}")

        for (i, j) in plus_cells:
            allocation[i][j] += theta
        for (i, j) in minus_cells:
            allocation[i][j] -= theta
```

```
229
230             iteration += 1
231
232         return allocation
233
234
235     # ------------------------------------------------------------------
236     # RUN
237     # ------------------------------------------------------------------
238     if __name__ == "__main__":
239         print("=" * 70)
240         print("TRANSPORTATION PROBLEM - INPUT DATA")
241         print("=" * 70)
242         print("Supply:", dict(zip(source_names, supply_orig)))
243         print("Demand:", dict(zip(dest_names, demand_orig)))
244         print("Cost matrix:")
245         for i, row in enumerate(cost):
246             print(f"  {source_names[i]}: {row}")
247
248         print("\n" + "=" * 70)
249         print("STEP 1: VOGEL'S APPROXIMATION METHOD (VAM) - Initial BFS")
250         print("=" * 70)
```

```
251         ibfs = vogel_approximation_method(supply_orig, demand_orig, cost)
252         print_table(ibfs, cost, "Initial Basic Feasible Solution (VAM):")
253         print(f"\nInitial (VAM) Total Transportation Cost = {total_cost(ibfs, cost)}")
254
255         print("\n" + "=" * 70)
256         print("STEP 2: MODI METHOD - Optimality Test & Improvement")
257         print("=" * 70)
258         optimal = modi_method(supply_orig, demand_orig, cost, ibfs)
259         print_table(optimal, cost, "\nOptimal Allocation (MODI):")
260         print(f"\nMinimum (Optimal) Total Transportation Cost = {total_cost(optimal, cost)}")
```

OUTPUT

```
======================================================================
TRANSPORTATION PROBLEM - INPUT DATA
======================================================================
Supply: {'S1': 50, 'S2': 60, 'S3': 50}
Demand: {'D1': 30, 'D2': 40, 'D3': 50, 'D4': 40}
Cost matrix:
  S1: [4, 6, 8, 8]
  S2: [6, 8, 6, 7]
  S3: [5, 7, 6, 8]

======================================================================
STEP 1: VOGEL'S APPROXIMATION METHOD (VAM) - Initial BFS
======================================================================
Step 1: Allocate 30 units to (S1 -> D1), cost/unit = 4
Step 2: Allocate 20 units to (S1 -> D2), cost/unit = 6
Step 3: Allocate 50 units to (S2 -> D3), cost/unit = 6
Step 4: Allocate 10 units to (S2 -> D4), cost/unit = 7
Step 5: Allocate 30 units to (S3 -> D4), cost/unit = 8
Step 6: Allocate 20 units to (S3 -> D2), cost/unit = 7
```

```
Initial Basic Feasible Solution (VAM):
               D1        D2        D3        D4
S1          30(4)     20(6)      -(8)      -(8)
S2           -(6)      -(8)     50(6)     10(7)
S3           -(5)     20(7)      -(6)     30(8)

Initial (VAM) Total Transportation Cost = 990

======================================================================
STEP 2: MODI METHOD - Optimality Test & Improvement
======================================================================

--- MODI Iteration 1 ---
u = ['S1:0', 'S2:0', 'S3:1']
v = ['D1:4', 'D2:6', 'D3:6', 'D4:7']
Opportunity costs (non-basic cells):
  S1-D3: 2
  S1-D4: 1
  S2-D1: 2
  S2-D2: 2
  S3-D1: 0
  S3-D3: -1
Entering cell: S3-D3 (opportunity cost = -1)
Closed loop: ['S3-D3', 'S3-D4', 'S2-D4', 'S2-D3', 'S3-D3']
Theta (max units to reallocate) = 30
```

```
--- MODI Iteration 2 ---
u = ['S1:0', 'S2:1', 'S3:1']
v = ['D1:4', 'D2:6', 'D3:5', 'D4:6']
Opportunity costs (non-basic cells):
  S1-D3: 3
  S1-D4: 2
  S2-D1: 1
  S2-D2: 1
  S3-D1: 0
  S3-D4: 1

All opportunity costs >= 0  ->  Optimal solution reached.


Optimal Allocation (MODI):
               D1        D2        D3        D4
S1          30(4)     20(6)      -(8)      -(8)
S2           -(6)      -(8)     20(6)     40(7)
S3           -(5)     20(7)     30(6)      -(8)

Minimum (Optimal) Total Transportation Cost = 960
```